



End-to-end LLMOps pipeline: requirements, challenges and prototype

May 6, 2026

Tutor:
Dr. Zhiming Zhao



Week 1

During the past week, I focused on estimating the GPU memory requirements for the local deployment environment of the thesis system.

I selected the Ministral 3 8B Instruct model as the current reference model because the Ministral 3 family is explicitly designed for compute- and memory-constrained applications [9]. The model follows a decoder-only Transformer architecture [14], with architectural specifications including 34 layers, hidden dimension 4096, and Grouped Query Attention (GQA) with 32 query heads and 8 key-value heads [9].

To estimate VRAM requirements, I used a formula adapted from the LocalLLM VRAM calculator [15], which is based on transformer memory scaling principles and KV cache analysis:

$$\text{VRAM} = (P \cdot b_w) + (0.55 + 0.08 \cdot P) + (B \cdot N \cdot 2 \cdot L \cdot (d/g) \cdot b_{kv}) \cdot 10^{-9}$$

where:

- P = number of model parameters,
- b_w = bytes per weight (quantization-dependent),
- B = batch size,
- N = context length,
- L = number of layers,
- d = hidden dimension,
- g = GQA grouping factor,
- b_{kv} = bytes per KV cache element.

The first term represents model weights, the second term captures runtime and activation overhead, and the third term corresponds to the KV cache memory, which grows linearly with context length and batch size. This linear scaling behavior is well-established in the literature on Transformer inference and KV cache optimization [7, 8, 16].

It is important to note that while the KV cache component of the formula is grounded in the Transformer architecture and supported by academic work, the overall formula is not fully derived from a single academic source. In particular, the overhead term is empirical and originates from system-level observations in inference frameworks rather than formal theoretical derivation. Therefore, the formula should be interpreted as a practical engineering approximation rather than an exact analytical model.

For this initial estimation, I assumed Q4_K_M quantization (approximately 0.5 bytes per weight), a context length of 60 000 tokens, and batch size $B = 1$. Under these conditions, the estimated VRAM requirement is approximately 9.4 GB.

Given that this estimate corresponds to a single batch and does not account for runtime variability (e.g., memory fragmentation, framework-specific overhead, or future scaling of context length), I provisionally set the target GPU requirement to 12 GB of VRAM. This provides sufficient headroom for stable deployment while remaining aligned with the constraints of available university infrastructure.

Future work will refine this estimate by evaluating different batch sizes, context lengths, and quantization strategies, as well as validating the theoretical estimate against empirical measurements.

Week 2

This week focused on establishing the observability layer of the pipeline across two distinct tiers: hardware resource monitoring and LLM-specific inference monitoring. This two-tier structure reflects the separation of concerns described in the LLMOps literature, where infrastructure-level signals and model-level signals require different instrumentation strategies and serve different diagnostic purposes [1, 6].



Hardware Monitoring Layer

The hardware monitoring layer was implemented using Prometheus with two exporters: `node-exporter` for host-level metrics and the NVIDIA DCGM exporter for GPU-specific telemetry. The resulting dashboard tracks four signals:

- **CPU utilisation (%)**: derived from `node_cpu_seconds_total` in idle mode, providing a baseline for host-level contention during inference.
- **System memory utilisation (%)**: computed from `node_memory_MemAvailable_bytes` relative to total memory, relevant for detecting memory pressure when the model is partially offloaded to RAM.
- **GPU utilisation (%)**: reported via `DCGM_FI_DEV_GPU_UTIL`, the primary signal for compute saturation during inference.
- **GPU memory used (GB)**: reported via `DCGM_FI_DEV_FB_USED`, directly validating the VRAM estimates established in Week 1 against empirical measurements at runtime.

The GPU memory panel is particularly significant in this context. The theoretical VRAM estimate of approximately 9.4 GB computed in Week 1 under Q4_K_M quantization, 60 000-token context length, and batch size $B = 1$ will be cross-validated against the `DCGM_FI_DEV_FB_USED` readings once experiments are run on hardware meeting the 12 GB provisioning target. Until then, the panel serves as the primary instrument for detecting out-of-memory conditions and quantifying the headroom available for scaling context length or batch size.

LLM Inference Monitoring Layer

The LLM-specific monitoring layer addresses a fundamental limitation of hardware metrics: GPU utilisation and memory consumption cannot distinguish between an idle model, a model processing a short prompt, and a model generating a long response. As noted by Pahune and Akhtar [11], the shift from MLOps to LLMOps introduces qualitatively different operational concerns, including non-deterministic outputs, prompt-sensitive behaviour, and inference-specific latency characteristics that standard infrastructure metrics cannot capture.

OpenLIT was selected as the instrumentation framework. It is OpenTelemetry-native, self-hostable, and provides direct integration with Ollama without requiring a custom proxy or token-counting wrapper. The framework is initialised with a single `openlit.init()` call and emits metrics conforming to the OpenTelemetry GenAI semantic conventions, which define a standardised schema for LLM observability signals. This choice reflects the principle that observability infrastructure for LLMs should be built on open, vendor-neutral standards [6].

The following metrics were selected for the inference dashboard on the basis of their relevance to the inference stage of the LLMOps lifecycle [1, 3]:

- **Time to first token (TTFT, p50/p95)**: the primary user-perceived latency signal for streaming inference, measuring the prefill phase duration.
- **Time per output token (p50/p95)**: a throughput quality metric capturing the decode phase; a divergence between p50 and p95 indicates tail degradation, potentially caused by context window pressure or thermal throttling detectable in the hardware layer.
- **Operation latency percentiles (p50/p95/p99)**: end-to-end request duration distribution. Average latency was explicitly excluded because it masks tail behaviour in non-deterministic systems [1].
- **Token count per hour (input and output)**: cumulative token volume per session, used to characterise prompt length stability across experiment runs and to monitor context window growth.
- **Token throughput (tokens/s)**: a rate metric meaningful only under sustained load; documented in the dashboard as interpretable primarily during automated benchmarking rather than interactive sessions.

- **Request error rate:** the fraction of requests resulting in an error, implemented with a zero-value fallback to distinguish healthy operation (flat line at zero) from absent data.

The two dashboard layers are intentionally complementary. A latency spike visible in the inference layer can be investigated in the hardware layer to determine whether it correlates with GPU memory pressure, thermal throttling, or CPU contention. This cross-layer correlation is the diagnostic workflow the pipeline is designed to support.

Implementation Notes

A practical difficulty encountered during implementation concerned the streaming nature of Ollama inference. Because metrics such as token counts and total latency are only available after the stream closes, all LLM-layer metrics are inherently post-hoc. This is not a limitation of the chosen framework but a fundamental property of streaming LLM inference: the information required to compute these metrics does not exist until the response is complete. This constraint is relevant when interpreting TTFT measurements, which are recorded at stream close rather than at the moment the first token is delivered to the client.

Due to hardware constraints on the development machine, no experimental results were collected this week. The pipeline is operational and validated for correct metric emission across both layers, but quantitative measurements will be deferred to runs on university infrastructure where the 12 GB VRAM provisioning target can be met without competing resource pressure.

Week 4

This week focused on surveying and classifying deployment strategies for LLM inference, with the goal of identifying which environments are relevant to the thesis pipeline and what observability requirements each one introduces. The analysis draws on the LLM Ops reference text [1], recent systems literature on LLM serving [2, 17], and cloud provider documentation. Three deployment strategies were selected for further investigation: single-node single-GPU deployment, cloud-based GPU instances, and Kubernetes-orchestrated multi-replica serving.

Strategy 1: Single-Node Single-GPU Deployment

In this strategy, the model is loaded entirely into a single GPU's VRAM on a single machine. The serving framework (in this pipeline, Ollama) handles request queuing and response streaming. This is the simplest deployment topology and the one currently implemented in the thesis pipeline with Ministral 8B under Q4_K_M quantization.

Aryan [1] notes that bare-metal setups offer the best raw performance for inference and are well suited for workloads where the model fits within a single device's memory. For on-premises deployments, the same source recommends Prometheus and Grafana for monitoring, which aligns with the observability stack already in place.

Advantages

- Full control over hardware and software stack, enabling unrestricted instrumentation at both the infrastructure and model layers.
- No variable operational costs; the only costs are hardware acquisition and energy consumption.
- Minimal operational complexity: no orchestration layer, no network partitioning, no distributed state to manage.
- Deterministic performance characteristics under single-user load, simplifying metric interpretation during controlled experiments.

Disadvantages

- Scalability is bounded by the GPU's VRAM and compute capacity. If the model does not fit, this strategy is not viable without quantization or architectural changes.
- No fault tolerance: a hardware failure results in complete service unavailability.
- Throughput is limited to what a single device can sustain, making this unsuitable for concurrent multi-user workloads.
- Capacity planning is static; there is no mechanism to adapt resources to demand fluctuations.

Observability Considerations

The monitoring stack established in Week 2 (Prometheus, Grafana, DCGM exporter, OpenLIT) covers both the infrastructure and model layers for this deployment. All metrics defined previously — TTFT, token throughput, error rate, GPU utilisation, VRAM consumption — are directly applicable without modification. This strategy serves as the observability baseline against which the requirements of other deployments can be compared.

Strategy 2: Cloud-Based GPU Instances

Cloud deployment involves provisioning GPU-equipped virtual machines from a provider such as AWS, Microsoft Azure, or Google Cloud Platform and deploying the containerised serving stack on them. The key distinction from the single-node strategy is elasticity: resources can be scaled up or down based on demand [1].

Each major provider offers GPU instance families suited to LLM inference. AWS provides EC2 instances with NVIDIA GPUs (e.g. the g5 series with A10G GPUs for smaller models, or p4d/p5 instances with A100 and H100 GPUs for larger workloads). Azure offers NC-series and ND-series VMs. GCP provides A2 and G2 machine types with A100 and L4 GPUs respectively [4]. All three providers support Infrastructure-as-Code (IaC) tools — AWS CloudFormation, Azure Resource Manager, Terraform — to make provisioning reproducible [1].

A distinctive feature of cloud deployment is auto-scaling. Services such as AWS Auto Scaling and Azure Virtual Machine Scale Sets can dynamically adjust instance counts based on metrics like GPU utilisation or request queue depth [1]. Cloud providers also offer cost-saving mechanisms such as spot instances (AWS) and preemptible VMs (GCP), which provide unused capacity at reduced prices but may be interrupted, requiring fault-tolerance mechanisms integrated with monitoring [1].

Advantages

- Low upfront cost with a pay-as-you-go pricing model, making it accessible without hardware investment.
- Elastic scalability: instances can be provisioned or terminated in response to demand, enabling cost-efficient handling of variable workloads.
- Managed infrastructure reduces operational burden for networking, power, cooling, and physical maintenance.
- Geographic distribution across availability zones and regions enables fault tolerance and latency optimisation for geographically distributed users.

Disadvantages

- Variable and potentially high operational costs, particularly for sustained GPU usage. As of 2026, on-demand H100 pricing ranges from approximately \$3.00/GPU/hour (GCP) to \$6.98/GPU/hour (Azure), though lower-tier GPUs suitable for 8B-parameter models are significantly cheaper [4].



- Reduced control over the hardware and software stack compared to on-premises deployment. GPU driver versions, hypervisor overhead, and multi-tenancy effects are not under the operator's control.
- Data privacy and regulatory constraints may prohibit transferring inference data to third-party infrastructure in certain domains.
- Infrastructure-layer observability is partially opaque: cloud-native monitoring tools (CloudWatch, Azure Monitor) provide standard metrics, but GPU-level telemetry such as DCGM metrics may not be available or may require additional configuration [1].

Observability Considerations

Cloud deployment introduces a split observability architecture. The infrastructure layer is partially covered by cloud-native tools (CloudWatch, Azure Monitor, GCP Operations Suite), but LLM-specific telemetry — TTFT, token throughput, per-request latency — still requires custom instrumentation such as the OpenLIT-based stack used in this pipeline [1]. Aryan [1] identifies three monitoring layers (client, application, infrastructure) and notes that cloud-native tools primarily address the infrastructure layer, leaving model-level observability as a gap that must be filled by the deployment operator.

Relevance to This Thesis

Although the pipeline may not be deployed on cloud infrastructure during the thesis timeline, this strategy is included because the pipeline's cost-estimation module will compute projected cloud costs for a given model configuration. By estimating VRAM requirements (as established in Week 1) and mapping them to cloud GPU instance types and regional pricing, the pipeline can produce cost projections that support deployment planning without requiring actual cloud provisioning.

Strategy 3: Kubernetes-Orchestrated Multi-Replica Serving

This strategy deploys multiple independent replicas of the same model, each running on its own GPU, behind a load balancer. It is distinct from distributed inference (where a single model is split across GPUs using tensor or pipeline parallelism [10, 13]): here, each replica holds a complete copy of the model and handles requests independently. Kubernetes manages replica lifecycle, health checking, and traffic distribution [1].

Aryan [1] provides deployment configurations using Kubernetes Deployments with configurable replica counts and Services for load balancing. The NVIDIA Triton Inference Server documentation [12] describes integration with Kubernetes Horizontal Pod Autoscaler (HPA) using Prometheus-scraped inference metrics (such as queue-to-compute ratio) to trigger scaling decisions. Google Cloud's best practices for LLM autoscaling recommend scaling on inference queue depth rather than CPU utilisation, since GPU workloads saturate compute while CPU remains low.

This approach is the natural horizontal scaling path from the single-node strategy: the model already fits on one GPU, so adding replicas increases throughput and availability without requiring model partitioning.

Advantages

- Horizontal scalability: throughput increases linearly with the number of replicas (assuming sufficient GPU resources and no shared bottleneck).
- Fault tolerance through redundancy: if one replica fails, Kubernetes automatically restarts it and redistributes traffic to healthy replicas [1].
- Independent scaling per model: in multi-model deployments, each model can scale based on its own demand without affecting others.



- Compatible with the existing containerised stack: the Docker Compose-based pipeline can be migrated to Kubernetes with relatively low effort.

Disadvantages

- Requires multiple GPUs, increasing hardware costs proportionally to the number of replicas.
- Introduces orchestration complexity: Kubernetes cluster management, GPU device plugin configuration, and autoscaler tuning add operational overhead [5].
- Model loading time creates cold-start latency when new replicas are provisioned; for multi-gigabyte models, this can take 30–120 seconds [5], during which the new replica cannot serve requests.
- Each replica maintains independent state (e.g. KV cache), so there is no memory sharing across replicas, leading to higher aggregate VRAM consumption compared to a single instance serving the same total throughput via batching.

Observability Considerations

Multi-replica deployment introduces monitoring requirements not present in the single-node setup. Per-replica health metrics must be collected and aggregated, including individual GPU utilisation, per-replica latency distributions, and load balancer traffic distribution. The observability stack must support both aggregate views (total throughput, system-wide error rate) and drill-down to individual replica metrics. Prometheus natively supports this through label-based filtering, which is compatible with the current pipeline’s metric collection approach. Additionally, autoscaler behaviour itself becomes a monitoring target: tracking scaling events, replica count over time, and the relationship between scaling triggers and latency improvements.

Deployment Strategies Noted but Not Explored

Two additional strategies were identified during the literature review but excluded from the thesis scope due to feasibility constraints. Third-party API-based inference (consuming models through provider APIs such as OpenAI or Anthropic) represents the most common deployment pattern in practice [1] but eliminates infrastructure-layer observability entirely, making it less relevant to this thesis’s focus on full-stack monitoring. Distributed multi-node inference using tensor and pipeline parallelism [10, 13] enables serving models that exceed a single GPU’s memory by partitioning the model across devices, but requires specialised hardware interconnects (NVLink, InfiniBand) and orchestration frameworks (Ray, DeepSpeed), placing it beyond the scope of a 13-week project.

References

- [1] A. Aryan. *LLMOps : managing large language models in production*. O’Reilly Media, 2025, p. 266. ISBN: 9781098154202.
- [2] G. Cai, R. Tian, L. Yang, Y. Jia, L. Li, and J. Wang. “Efficient Inference for Edge Large Language Models: A Survey”. In: *Tsinghua Science and Technology* 31.3 (June 2026), pp. 1365–1380. ISSN: 1007-0214. DOI: 10.26599/TST.2025.9010166. URL: <https://www.sciopen.com/article/10.26599/TST.2025.9010166>.
- [3] M. Chernigovskaya, D. Singh Walia, K. Neumann, A. Hardt, A. Nahhas, and K. Turowski. “Towards a Standardized Business Process Model for LLMOps”. In: (2025). DOI: 10.5220/0013377700003929.
- [4] *Cloud GPU Pricing Comparison: AWS Vs. Azure Vs. GCP For AI*. URL: <https://www.cloudzero.com/blog/cloud-gpu-pricing-comparison/>.

- *****
- [5] *Configure autoscaling for LLM workloads on GPUs with Google Kubernetes Engine (GKE) — GKE AI/ML — Google Cloud Documentation*. URL: <https://docs.cloud.google.com/kubernetes-engine/docs/how-to/machine-learning/inference/autoscaling>.
 - [6] J. Diaz-De-Arcaya, J. López-De-Armentia, R. Miñón, I. L. Ojanguren, and A. I. Torre-Bastida. “Large Language Model Operations (LLMOps): Definition, Challenges, and Lifecycle Management”. In: *2024 9th International Conference on Smart and Sustainable Technologies, SpliTech 2024* (2024). DOI: 10.23919/SPLITECH61897.2024.10612341. URL: <https://ieeexplore.ieee.org/abstract/document/10612341>.
 - [7] Z. Guo, H. Kamigaito, and T. Watanabe. “Attention Score is not All You Need for Token Importance Indicator in KV Cache Reduction: Value Also Matters”. In: *EMNLP 2024 - 2024 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference* (June 2024), pp. 21158–21166. DOI: 10.18653/v1/2024.emnlp-main.1178. URL: <https://arxiv.org/pdf/2406.12335>.
 - [8] J.-H. Kim, J. Kim, S. Kwon, J. W. Lee, S. Yun, and H. O. Song. “KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction”. In: (May 2025). URL: <https://arxiv.org/pdf/2505.23416>.
 - [9] A. H. Liu et al. “Ministral 3”. In: (Jan. 2026). URL: <https://arxiv.org/pdf/2601.08584>.
 - [10] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia. “Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM”. In: *International Conference for High Performance Computing, Networking, Storage and Analysis, SC* (Nov. 2021). ISSN: 21674337. DOI: 10.1145/3458817.3476209. URL: [/doi/pdf/10.1145/3458817.3476209?download=true](https://doi/pdf/10.1145/3458817.3476209?download=true).
 - [11] S. Pahune and Z. Akhtar. “Transitioning from MLOps to LLMOps: Navigating the Unique Challenges of Large Language Models”. In: *Information 2025, Vol. 16, Page 87* 16.2 (Jan. 2025), p. 87. ISSN: 2078-2489. DOI: 10.3390/INF016020087. URL: <https://www.mdpi.com/2078-2489/16/2/87/htm%20https://www.mdpi.com/2078-2489/16/2/87>.
 - [12] *Scaling LLMs with NVIDIA Triton and NVIDIA TensorRT-LLM Using Kubernetes — NVIDIA Technical Blog*. URL: <https://developer.nvidia.com/blog/scaling-llms-with-nvidia-triton-and-nvidia-tensorrt-llm-using-kubernetes/>.
 - [13] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro. “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism”. In: (Sept. 2019). URL: <https://arxiv.org/pdf/1909.08053>.
 - [14] A. Vaswani, G. Brain, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems* 30 (2017).
 - [15] *VRAM Calculator for Local Open Source LLMs - Accurate Memory Requirements 2025 — LocalLLM.in*. URL: <https://localllm.in/blog/interactive-vram-calculator>.
 - [16] Y. Xu, N. K. Khaira, and T. Singh. “KV Cache Optimization Strategies for Scalable and Efficient LLM Inference”. In: (Mar. 2026). URL: <https://arxiv.org/pdf/2603.20397>.
 - [17] R. Zhen, J. Li, Y. Ji, Z. Yang, T. Liu, Q. Xia, X. Duan, Z. Wang, B. Huai, and M. Zhang. *Taming the Titans: A Survey of Efficient LLM Inference Serving*. 2025. URL: <https://aclanthology.org/2025.inlg-main.32/>.
- *****